Lab Session 11

Agile Sprint Planning Using Trello

1. Introduction

In modern software development, Agile methodology is widely used to manage projects that require flexibility, rapid delivery, and collaboration. Unlike traditional waterfall models, Agile emphasizes iterative development, frequent feedback, and adaptive planning.

The Scrum framework is one of the most popular Agile frameworks. It organizes work into time-boxed iterations called sprints, where the team delivers working software incrementally.

The goal of this lab is to teach students how to plan and manage a sprint using Scrum principles, visualize tasks using Trello, and monitor progress using priority and burn-down charts.

2. Learning Objectives

By completing this lab, students will learn to:

1. Understand **Agile methodology and Scrum framework**
2. Define **Sprint Goals** aligned with project objectives
3. Select **user stories** from a product backlog and prioritize them
4. Break user stories into actionable tasks
5. Assign tasks to team members effectively
6. Estimate effort using hours or story points
7. Set up and manage a Trello Sprint Board
8. Draw priority charts and burn-down charts
9. Conduct a reflection on sprint planning, task management, and coordination

3. Agile Methodology

Agile methodology is a **set of principles for software development** that prioritize:

- **Individuals and interactions** over rigid processes and tools
- **Working software** over extensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a strict plan

Key Benefits of Agile:

- Faster delivery of functional software
- Improved customer satisfaction through frequent feedback
- Adaptable to changing requirements
- Increased team collaboration and accountability

Agile Practices:

1. **Iterative Development:** Break the project into small increments delivered in sprints.
2. **Continuous Feedback:** Engage stakeholders regularly to validate functionality.
3. **Adaptive Planning:** Reassess priorities after each iteration.
4. **Collaborative Teams:** Cross-functional teams work together on design, development, and testing.

4. Scrum Framework

Scrum is a lightweight framework for managing complex projects using Agile principles.

4.1 Scrum Roles

Role	Responsibilities
Product Owner	Manages the product backlog, prioritizes stories, ensures team builds the right features
Scrum Master	Facilitates Scrum processes, removes obstacles, ensures team follows Agile practices
Development Team	Implements and tests tasks, self-organizing, responsible for delivering potentially shippable increments

4.2 Scrum Artifacts

1. **Product Backlog** – A complete list of features, enhancements, and bug fixes needed for the product.
2. **Sprint Backlog** – Subset of the product backlog selected for a sprint.
3. **Increment** – Completed, functional product delivered at the end of a sprint.

4.3 Scrum Ceremonies

1. **Sprint Planning:** Define sprint goal, select stories, break them into tasks, assign work.
2. **Daily Stand-up:** Quick daily meeting to review progress and obstacles.
3. **Sprint Review:** Demonstrate the completed increment to stakeholders.
4. **Sprint Retrospective:** Reflect on the sprint to improve team processes.

5. Sprint Planning

Sprint Planning ensures that the development team knows:

- **What to build** (Sprint Goal)
- **How to build it** (Tasks, assignments, and dependencies)
- **How long it will take** (Effort estimation)

5.1 Steps in Sprint Planning

1. **Define Sprint Goal** – Clear objective for the sprint, aligned with product vision.
2. **Select User Stories** – Choose items from the product backlog achievable within the sprint.
3. **Prioritize Stories** – Identify high-value stories first.
4. **Break Stories into Tasks** – Smaller tasks are easier to estimate and assign.
5. **Estimate Effort** – Use hours or story points for planning capacity.
6. **Assign Tasks** – Allocate work based on team members' expertise.
7. **Plan Sprint Board** – Visualize tasks in **To Do, In Progress, Done** columns.
8. **Visualize Progress** – Priority charts and burn-down charts track performance.

6. Product Backlog

Before we can begin a sprint, we need to **prepare a product backlog**.

What is a Product Backlog?

- A product backlog is a **living document** listing all features, enhancements, bug fixes, and tasks that could be done for a project.
- Each item is written as a **user story**, e.g., "As a user, I want to log in so that I can access my account."
- Stories should be **prioritized** by value, risk, and dependencies.

Breaking Stories into Tasks:

- Each story should be broken into **smaller, manageable tasks**.
- Tasks are easier to **estimate, assign, and track** than full user stories.
- Example tasks for a "Login" story:
    - o Design login form UI
    - o Implement backend authentication
    - o Connect UI with backend
    - o Test login functionality

Example Product Backlog Table

Students create the **Product Backlog Table** in Word.

ID	User Story	Description	Priority
US1	User Login	Implement login functionality	High
US2	User Registration	Sign-up form & validation	High
US3	Profile Management	Edit profile details	Medium
US4	Dashboard	Display recent activity	Medium
US5	Notifications	Alerts for updates	Low

7. Breaking Stories into Tasks

- Breaking stories into tasks ensures clarity, accurate estimation, and easier tracking.
- Each task should ideally be **completable within 1–2 days**.

Example: US1 – User Login

Task ID	**Task Description**	**Assigned To**	**Estimated Hours**	**Status**
T1.1	Design login form UI	Front-end	3	To Do
T1.2	Implement backend authentication	Back-end	4	To Do
T1.3	Connect UI with backend	Front-end	2	To Do
T1.4	Test login functionality	QA	2	To Do

8. Priority Chart

- Priority charts **visualize which stories/tasks are most important**.
- Helps the team focus on **high-value features first**.

Students count tasks according to priority.

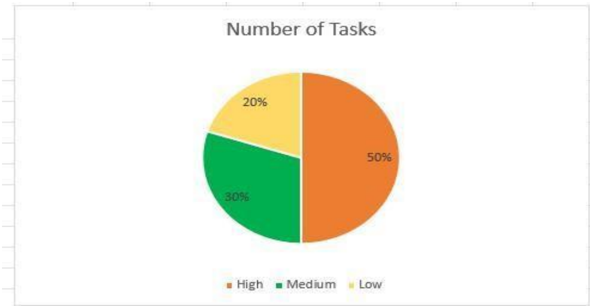
Steps

1. Open **Microsoft Excel**
2. Enter the table
3. Select the table
4. Click **Insert**
5. Choose **Bar Chart or Pie Chart**

Format:

X-axis → Priority
Y-axis → Number of Tasks

Priority	**Number of Tasks**
High	5
Medium	3
Low	2



9. Trello Sprint Board

- Trello is a **Kanban-style board** to manage tasks.
- Columns (lists) represent **workflow stages**.

Students manage task using trello.

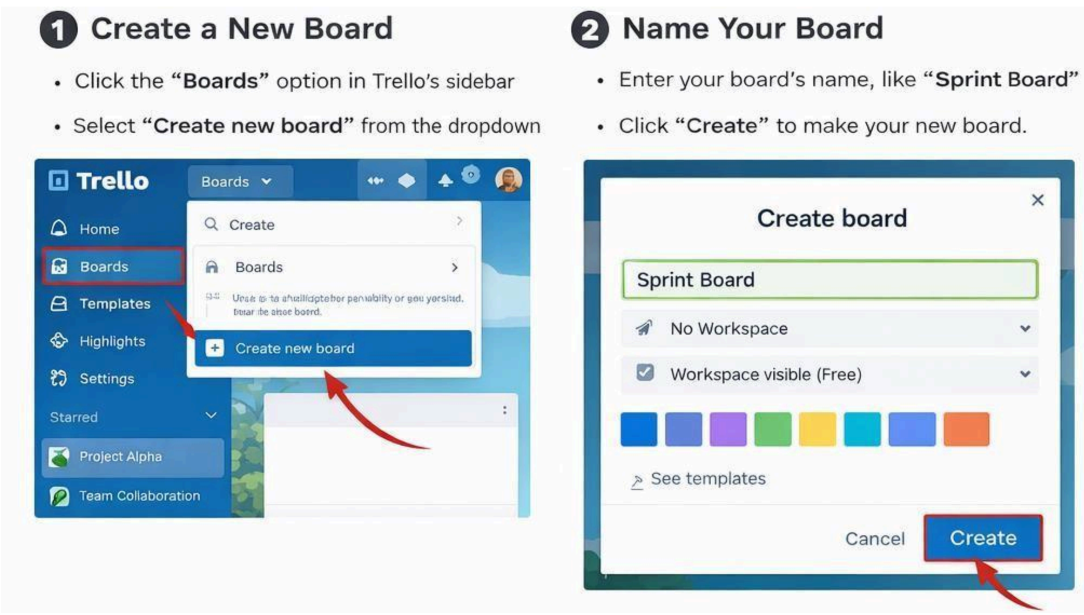
Step-by-step Instructions:

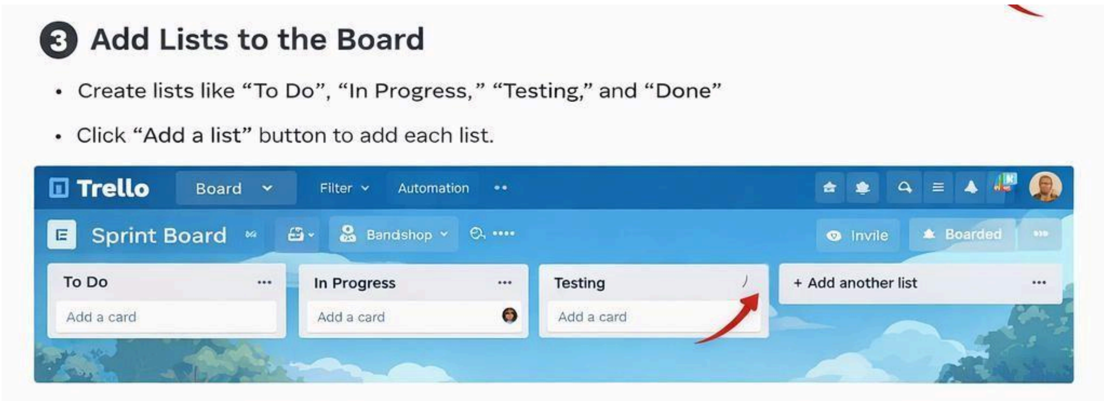
1. Create a new Trello board.
2. Create **columns**: To Do | In Progress | Done
3. Add **cards** for each task.
4. Assign **team members** to each card.
5. Move cards from **To Do** → **In Progress** → **Done** as work progresses.

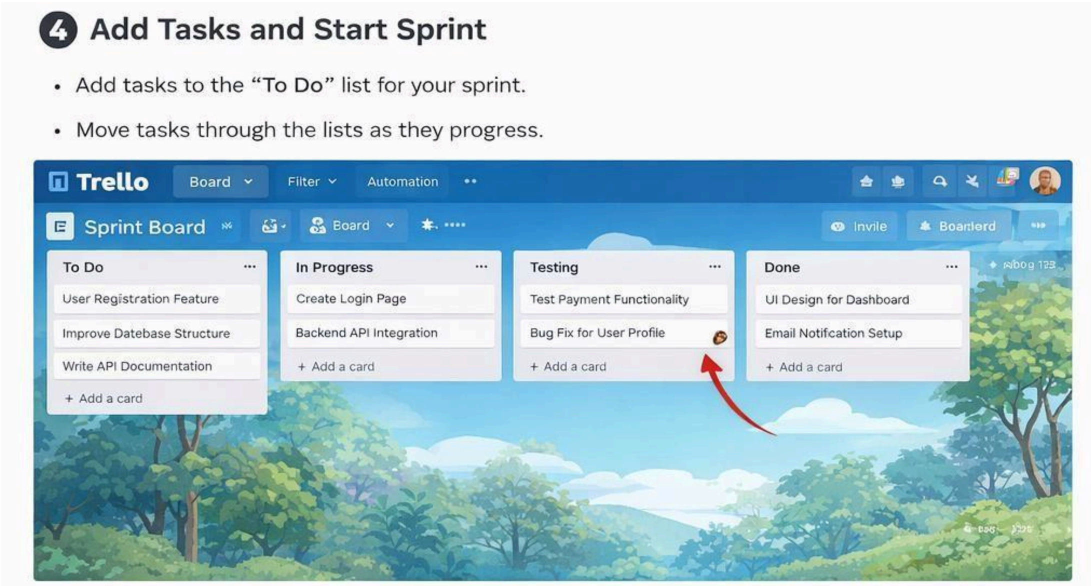
Example Layout:

- **To Do:** T1.1, T1.2, T2.1
- **In Progress:** T1.3
- **Done:** T1.4

Tip: Use **labels** to show priority, story type, or department.







10. Burn-down Chart

- Burn-down charts track **remaining work vs sprint days**.
- Helps monitor progress and identify if the team is on schedule.

Students track **remaining work each day**.

Step-by-step Instructions:

1. Record **estimated hours** for each task in Trello (custom field or card description).
2. At the end of each day, calculate **remaining hours**.
3. Create a table in Excel or Sheets:

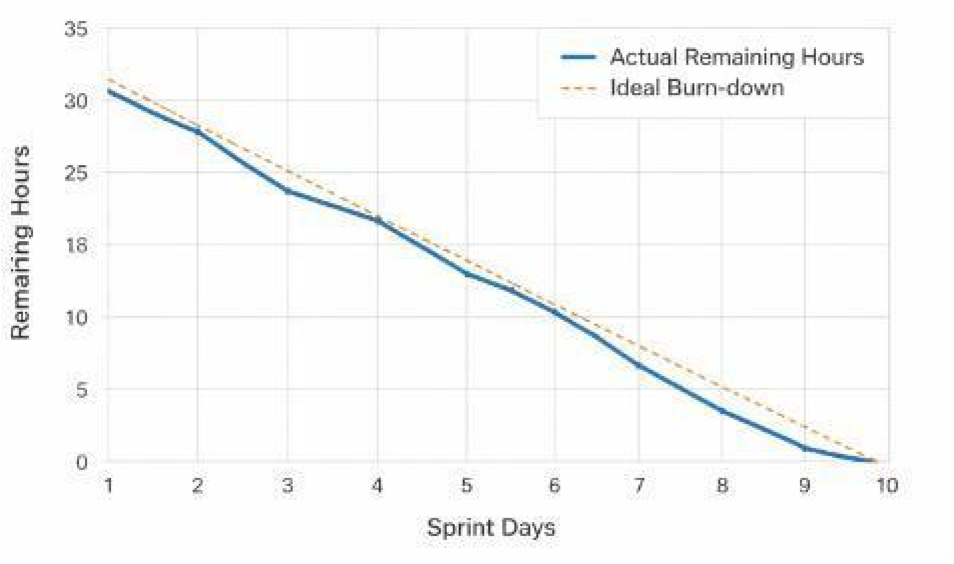
Day	Remaining Hours
1	32
2	28
3	25
4	20
5	16
6	12

7	9
8	5
9	2
10	0

4. Insert a **line chart**:

- X-axis = Sprint Days
- Y-axis = Remaining Hours
- Plot actual progress against ideal trend line

Optional: Use Trello Power-Ups like Corrello for automated burn-down charts.



EXERCISE

Project: Simple Calculator Web Application

Instructions:

1. Define **Sprint Goal**
2. Prepare **Product Backlog**: Addition, Subtraction, Multiplication, Division, Input Validation, Error Handling, Responsive UI
3. Break each story into tasks, estimate effort, assign team members
4. Create **Trello Sprint Board**
5. Draw **priority chart** and **burn-down chart**

Deliverables:

- Sprint Goal document
- Sprint Backlog table with tasks, assignments, estimated hours
- Trello board screenshot
- Priority and burn-down charts
- Reflection paragraph

Task 1

Project: Simple Calculator Web Application Instructions:

1. Define Sprint Goal
2. Prepare Product Backlog: Addition, Subtraction, Multiplication, Division, Input Validation, Error Handling, Responsive UI
3. Break each story into tasks, estimate effort, assign team members
4. Create Trello Sprint Board
5. Draw priority chart and burn-down chart

Deliverables:

- Sprint Goal document
- Sprint Backlog table with tasks, assignments, estimated hours
- Trello board screenshot
- Priority and burn-down charts
- Reflection paragraph

1. Sprint Goal

To develop a fully functional, responsive web-based calculator that performs addition, subtraction, multiplication, and division while ensuring robust input validation and error handling.

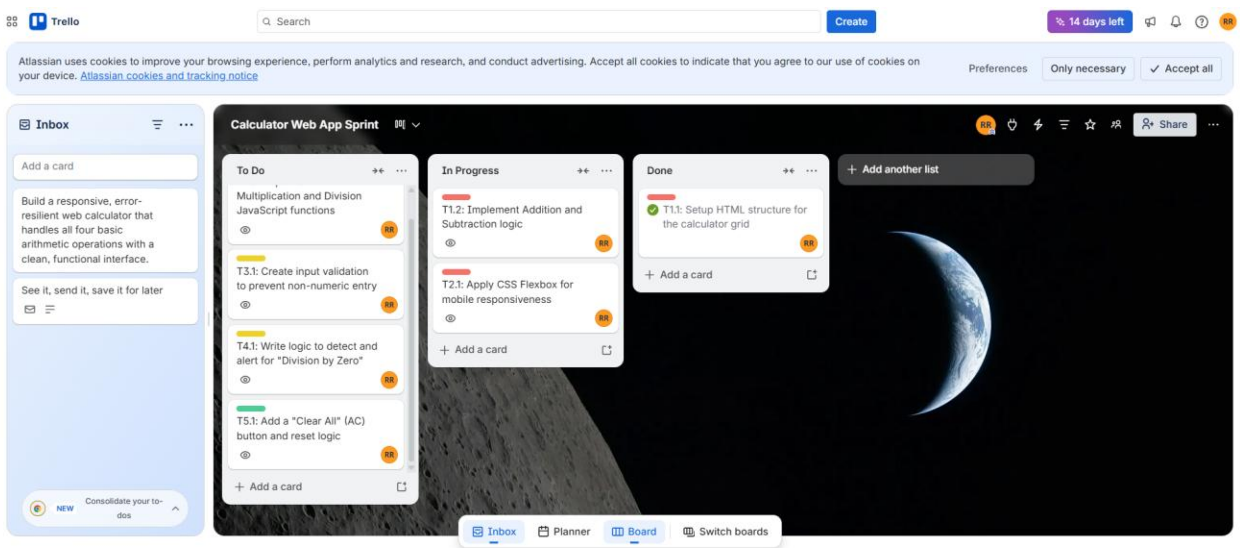
2. Product Backlog

ID	User Story	Description	Priority
US1	Basic Arithmetic	Perform addition, subtraction, multiplication, and division.	High
US2	Responsive UI	Ensure the calculator works on mobile and desktop screens.	High
US3	Input Validation	Prevent users from entering invalid characters or symbols.	Medium
US4	Error Handling	Manage errors like "Division by Zero" without crashing.	Medium
US5	Basic Functions	Implement a "Clear" (AC) and equals (=) button.	Low

3. Sprint Backlog (Task Breakdown)

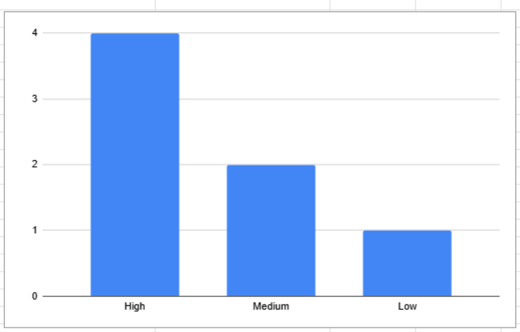
Task ID	Task Description	Assigned To	Est. Hours
T1.1	Setup HTML structure for the calculator grid	**Raazia**	2
T1.2	Implement Addition and Subtraction JavaScript functions	**Waheeba**	3
T1.3	Implement Multiplication and Division JavaScript functions	**Waheeba**	3
T2.1	Apply CSS Flexbox for mobile responsiveness	**Raazia**	3
T3.1	Create input validation to prevent non-numeric entry	**Aiza**	2
T4.1	Write logic to detect and alert for "Division by Zero"	**Aiza**	2
T5.1	Add a "Clear All" (AC) button and reset logic	**Waheeba**	1

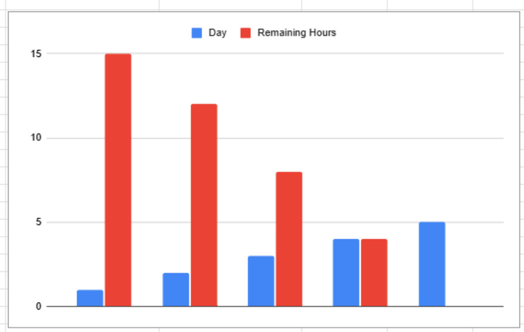
4. Trello Board



5. Priority and burn-down charts

Priority	Number of Tasks	Day	Remaining Hours
High	4	1	15
Medium	2	2	12
Low	1	3	8
		4	4
		5	0





6. Final Reflection

This lab demonstrated how Agile planning transforms a broad project like a 'Calculator Web App' into manageable, prioritized steps. By breaking user stories into specific tasks, we were able to identify that core arithmetic logic (High Priority) must be completed before visual polishing (Low Priority). Using Trello and Burn-down charts provided a clear roadmap, ensuring that time-estimation was realistic and that project bottlenecks were visible before they caused delays